

Reproducing Ithemal: A Learned Cost Model for x86 Programs

Chi Gui
University of Illinois at
Urbana-Champaign
Urbana, IL, USA

Runhua Li
University of Illinois at
Urbana-Champaign
Urbana, IL, USA

Yuxuan Ma
University of Illinois at
Urbana-Champaign
Urbana, IL, USA

Abstract

Accurate modeling of instruction throughput is critical for performance analysis and compiler optimizations on modern CPUs. Traditional analytical cost models often fail to generalize across architectures or complex microarchitectural interactions. In this project, we reproduce Ithemal, a learned cost model that leverages deep neural networks to estimate the throughput of x86 basic blocks. We train the model using the BHive benchmark suite, a large corpus of hand-crafted instruction sequences and their measured performance on Intel architectures. Our work evaluates the training process, model accuracy, and implementation challenges, providing insights into the replicability of learned cost models and the viability of machine learning approaches in compiler design.

Keywords: Machine Learning for Compilers, Cost Modeling, Performance Prediction, Ithemal, BHive Benchmark, x86, Instruction Throughput, Deep Neural Networks

1 Problem Statement and Motivation

Modern compilers rely heavily on cost models to estimate the performance of basic blocks and guide optimization decisions such as instruction scheduling and register allocation. Traditional cost models, such as those based on analytical heuristics (e.g., LLVM-MCA or Intel’s IACA), are often brittle, architecture-specific, and difficult to maintain. These models typically struggle to capture the true complexity of modern out-of-order superscalar processors, especially when dealing with subtle microarchitectural behaviors such as port pressure, instruction fusion, and memory aliasing.

To address these limitations, the *Ithemal* project proposes a data-driven approach to basic block throughput estimation using deep learning. By learning directly from empirical performance measurements collected on real hardware (e.g., Intel Haswell, Skylake, Ivy Bridge), Ithemal replaces hand-tuned heuristics with a neural network that can generalize across instruction patterns and processor architectures.

The goal of this project is to replicate the Ithemal framework and demonstrate that a machine learning-based cost model can achieve highly accurate throughput prediction across multiple x86 microarchitectures. Through model training, evaluation, and cross-architecture testing using the BHive

dataset, we aim to validate the original findings and gain insight into the effectiveness and portability of learned cost models in compiler toolchains.

2 Related Work

Traditional cost models for instruction throughput prediction, such as Intel’s IACA [1] and LLVM-MCA [2], use hand-crafted rules to estimate performance characteristics based on instruction semantics and microarchitectural constraints. While these models are widely used in compiler toolchains, they require significant manual effort to maintain and often fall short in capturing complex interactions like port contention, data dependencies, and speculative execution effects on modern superscalar processors.

To address these limitations, several works have explored machine learning approaches for performance modeling. Early attempts include statistical models trained on hardware counters [4], which provided limited generalization across code structures. Deep learning-based solutions, such as Ithemal [6], represent a more powerful alternative. Ithemal employs a hierarchical recurrent neural network to encode instruction sequences and directly regress throughput values, trained using real latency measurements from the BHive benchmark suite [5].

Other contemporary efforts have extended this paradigm. For example, Aamer et al. [3] introduced a graph-based neural model that incorporates dependency edges between instructions, while Prodigy [7] proposes a GNN-based architecture that adapts to multiple downstream code analysis tasks, including cost modeling. Unlike these works, which often propose novel architectures, our project focuses on reproducing Ithemal faithfully to evaluate its robustness and replicability as a foundational baseline in learned cost modeling.

3 Design Overview

Our goal is to faithfully reproduce the Ithemal framework, a learned cost model that predicts the throughput of x86 basic blocks using deep neural networks. The original system comprises three major components: (1) a data processing pipeline that prepares training data from the BHive benchmark suite, (2) a neural network architecture for sequence modeling of instructions, and (3) a training and validation loop that optimizes the model to predict throughput values.

We preserved the original design of Ithema’s hierarchical LSTM model, which processes each instruction as a sequence of tokens (e.g., opcode, operands, memory specifiers) and aggregates them into a representation of the full basic block. These tokenized sequences are passed through an instruction-level LSTM followed by a block-level LSTM to produce a throughput prediction. The model is trained using mean squared error loss on real-world measurements from BHive.

To maximize compatibility and simplify dependency management, we chose to reproduce the system in a legacy Python 2 environment. This decision enabled us to build and run the full pipeline, including training, validation, and tokenization tools, without needing to modify core logic or port deprecated modules. All data processing scripts, tokenizer binaries, and training code were executed using Python 2.7 with manually resolved dependencies.

The overall pipeline follows the structure described in the original Ithema paper. We begin by converting the BHive CSV dataset into a serialized format suitable for training. This involves decoding hex-encoded instruction bytes into a sequence of instruction tokens, constructing a basic block representation, and associating it with a measured throughput value. Once the dataset is serialized, we train the model using the Ithema-provided multi-process training coordinator. After training, we evaluate model accuracy using the default validation routine and report metrics based on held-out test data.

Our design stays close to the original implementation to assess the reproducibility of the Ithema system and to identify which parts of the workflow are most sensitive to environment changes or data quality.

4 Implementation and Evaluation

4.1 Implementation

We trained the Ithema model using its official PyTorch-based multi-process training framework. The model was trained for 10 epochs with a batch size of 4 on 4 parallel trainer processes, each utilizing 4 CPU threads. We used the Adam optimizer with private moment tracking, an initial learning rate of 0.1, a learning rate decay schedule applied at the end of each epoch, and a momentum parameter of 0.9. No weight decay or Nesterov momentum was applied. The training data was partitioned evenly among the trainers (1,000 samples per trainer).

For model architecture, we used an embedding size of 256 and a hidden layer size of 256. The RNN component was enabled with an LSTM cell (`-rnn-lstm`) and a multi-scale hierarchical structure (`-rnn-multiscale`). Skip connections, token-to-token connections, and learned initial states were disabled. We adopted the max-reduction strategy (`-dag-max-reduction`) for aggregating information across the DAG, without applying additional nonlinearity layers.

The model predicted the raw execution time rather than its logarithm (`-predict-log` disabled). Linear embeddings and residual connections were enabled, while DAG-RNNs were retained.

The training was conducted using the official dataset, with instruction memory effects preserved (`-no-mem` disabled) and without any edge ablation strategies.

4.2 Experimental Setup

We conducted our experiments using the BHive benchmark dataset, which provides basic blocks and corresponding throughput measurements collected from real hardware across multiple x86 microarchitectures. In this project, we used the official datasets for Haswell, Ivy Bridge, and Skylake. For each architecture, we generated a separate training file in Ithema’s serialized `.data` format.

Three separate models were trained, one per architecture, using 10 training epochs each. We used the original Ithema training loop with its default multi-process configuration to ensure consistency across experiments. To evaluate portability, we applied each trained model not only to its own architecture’s dataset but also to the datasets of the other two architectures.

To evaluate model performance, we used the metrics reported in the original Ithema paper: normalized L1 error, Spearman rank correlation, and Pearson correlation. During testing, we compared predicted throughput values to ground-truth hardware measurements provided by BHive. Additionally, we visualized prediction accuracy using scatter plots and histograms to highlight model behavior across different throughput ranges.

4.3 Challenge

We closely followed the official Ithema codebase, which is implemented in Python 2 and relies on legacy packages that are no longer compatible with modern Python environments. One of the major challenges we faced during implementation was the incompatibility of the code with Python 3. This prevented us from running the training pipeline directly in a modern system.

To address this, we created a Conda environment with Python 2.7 and manually installed all required dependencies, including a compatible version of PyTorch. We also compiled the tokenizer from source using the original CMake-based build system. This setup allowed us to run the original training and evaluation scripts without any modification to the codebase.

We used the official BHive benchmark CSV files for each microarchitecture (Haswell, Skylake, Ivy Bridge) and preprocessed them into `.data` files using the original data loader. Each `.data` file contains serialized tuples of basic block ID, measured throughput, disassembled code, and tokenized instruction sequences. These files served as the direct input to the model training and prediction framework.

4.4 Results

4.4.1 Evaluation on Intel Haswell (HSW). We begin with the Haswell (HSW) microarchitecture, evaluated on 59,871 samples. As shown in Table 1, the model achieves an MAE of 84.02 cycles and an RMSE of 1580.95 cycles. The R^2 value is relatively low (0.0685), but the Spearman rank correlation ($\rho = 0.9283$) indicates strong preservation of relative throughput order.

Figure 1 visualizes prediction density with a hexbin plot. Most values align near the $y = x$ line, with three boxed regions showing clusters of high absolute percentage error. These typically contain blocks with memory aliasing or infrequent instructions. Figure 2 presents throughput predictions binned by actual value, showing stable accuracy for common low-to-mid throughput blocks and underestimation for high-throughput ones, which are rare in the dataset.

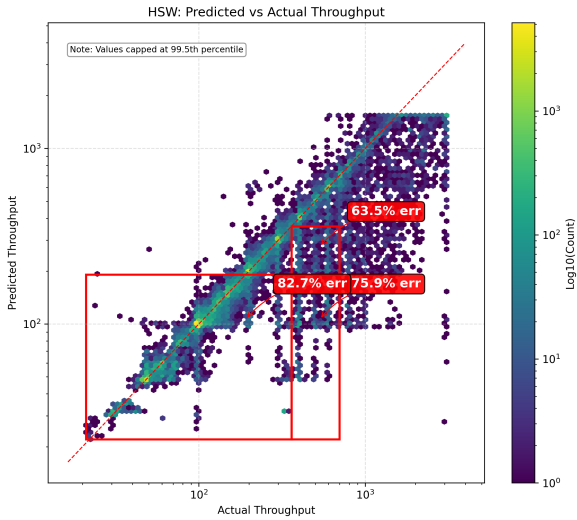


Figure 1. HSW: Predicted vs Actual Throughput (hexbin, log-log). High-error clusters are boxed.

4.4.2 Evaluation on Intel Ivy Bridge (IVB). IVB results (59,496 samples) are shown in Table 1. Compared to HSW, the model achieves slightly lower MAE (82.09) and RMSE (1197.50), and a higher R^2 (0.116) and Spearman ρ (0.9335). Figure 3 shows dense clustering along $y = x$, with annotated regions of systematic underprediction. Figure 4 confirms accurate performance across most bins; outliers primarily affect sparsely populated high-throughput regions.

4.4.3 Evaluation on Intel Skylake (SKL). Skylake (SKL) results, based on 58,722 samples, are also summarized in Table 1. It achieves the best performance with MAE of 75.34, RMSE of 901.72, and the highest R^2 (0.171). Spearman ρ remains high at 0.9281. As shown in Figure 5, predictions closely track the diagonal except in a few high-error regions.

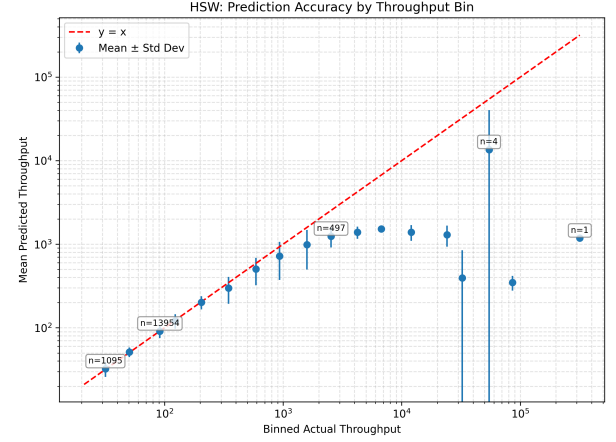


Figure 2. HSW: Prediction Accuracy by Throughput Bin. Error bars denote standard deviation.

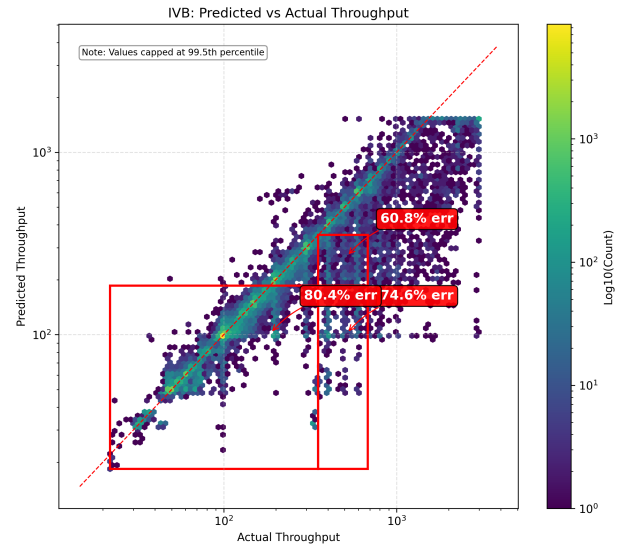


Figure 3. IVB: Predicted vs Actual Throughput (Hexbin). High-error regions boxed.

Figure 6 confirms good alignment between predicted and actual throughput in most bins.

Table 1. Evaluation Metrics Across Architectures

Architecture	MAE	RMSE	R^2	Spearman ρ
HSW	84.02	1580.95	0.0685	0.9283
IVB	82.09	1197.50	0.1160	0.9335
SKL	75.34	901.72	0.1710	0.9281

4.4.4 Cross-Architecture Comparison. From Table 1, we see that SKL consistently outperforms the other architectures, with lowest prediction error and highest correlation

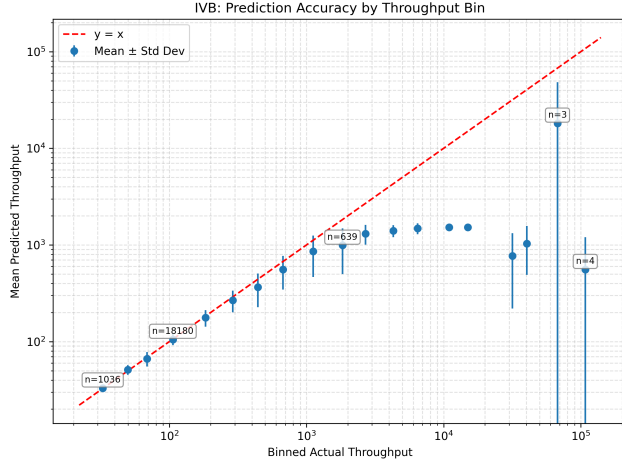


Figure 4. IVB: Prediction Accuracy by Throughput Bin.

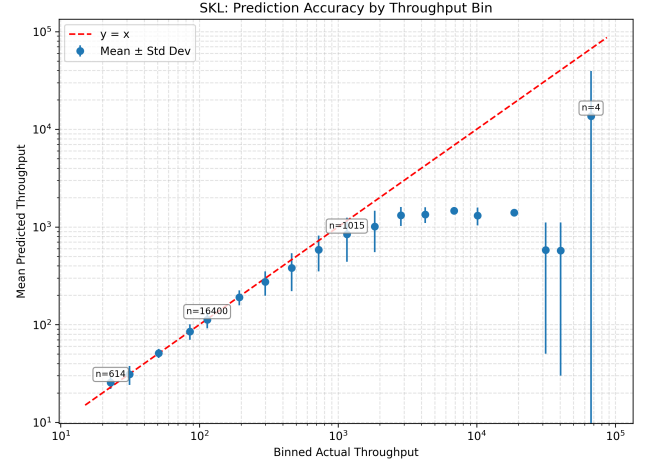


Figure 6. SKL: Prediction Accuracy by Throughput Bin.

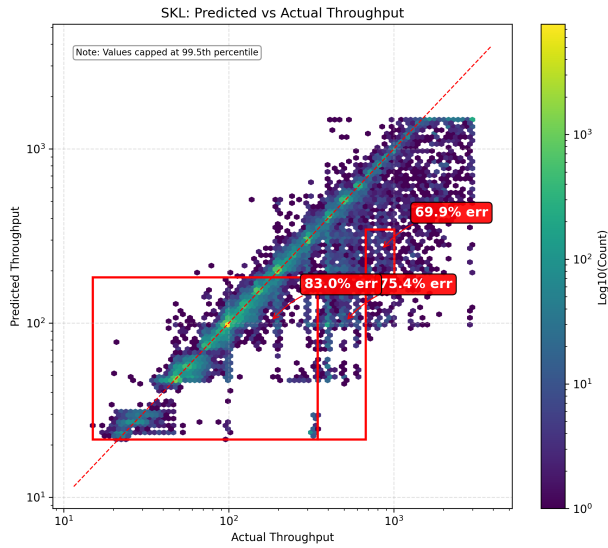


Figure 5. SKL: Predicted vs Actual Throughput (Hexbin). High-error regions boxed.

scores. HSW shows the weakest R^2 and highest RMSE, likely due to outlier-heavy instruction patterns. Despite these differences, all models achieve $\rho \approx 0.93$, showing reliable rank ordering, a key requirement for compiler heuristics.

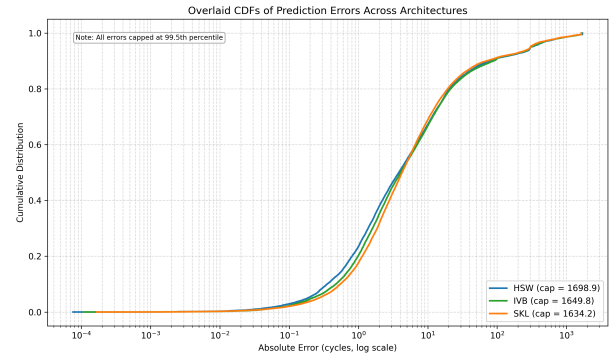


Figure 7. CDFs of absolute prediction errors across architectures (99.5% capped).

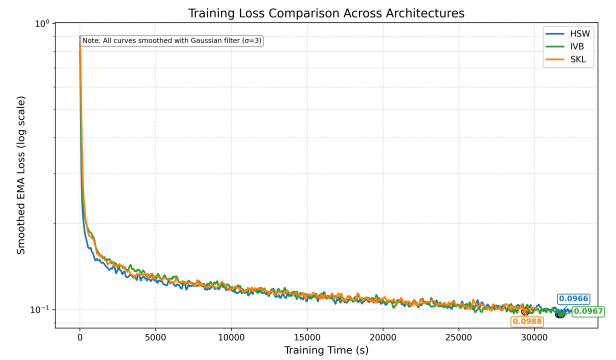


Figure 8. Smoothed EMA training loss across architectures (log scale).

References

- [1] [n. d.]. Intel Architecture Code Analyzer. <https://software.intel.com/en-us/articles/intel-architecture-code-analyzer>. Accessed: 2025-05-15.
- [2] [n. d.]. LLVM Machine Code Analyzer. <https://llvm.org/docs/CommandGuide/llvm-mca.html>. Accessed: 2025-05-15.
- [3] Farhan Aamer, Andreas Kipf, Thomas Müller, Benedikt Radke, Alfons Kemper, and Thomas Neumann. 2020. Learning a neural cost model for query optimization with pipelines. In *CIDR*.
- [4] Kenneth Hoste and Lieven Eeckhout. 2006. Performance prediction based on inherent program similarity. In *Proceedings of the 15th international conference on Parallel architectures and compilation techniques*. ACM, 114–122.
- [5] Chathuranga Mendis, Saman Amarasinghe, and Michael Carbin. 2019. BHive: A benchmark suite and measurement framework for validating x86-ISA performance models. In *2019 IEEE International Symposium on Workload Characterization (IISWC)*. IEEE, 108–119.
- [6] Chathuranga Mendis, Saman Amarasinghe, and Michael Carbin. 2019. Ithemal: Accurate, portable and fast basic block throughput estimation using deep neural networks. In *International Conference on Machine Learning*. 4505–4515.
- [7] Junru Zhao, Yifan Wang, and Daniel Sanchez. 2021. Prodigy: Neural prediction of microarchitectural performance. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 29–42.